AI Intern Task: Low-Latency Voice Assistant with RAG (India, Production-Grade)

Problem Statement

Build a **real-time AI voice assistant** that interacts via microphone, retrieves answers using RAG, and responds with speech under strict latency constraints.

Objective

- real-time speech-to-speech interaction
- retrieval-augmented responses
- backend action capability
- **sub-2 second response latency (mandatory)**

Free Credits Setup (MANDATORY)

Use Google Cloud free credits:

Watch setup guide

Setup Requirements

1. Create Google Cloud account
2. Activate $300 credits
3. Enable billing
4. Enable Vertex AI API
5. Set region:

```
asia-south1 (Mumbai)
```

Performance Targets

- first audio response: ≤ 1.5 sec (target), ≤ 2 sec max
- LLM first token: ≤ 800 ms
- RAG retrieval: ≤ 150 ms
- no pauses > 500 ms

REQUIRED TECH STACK

Core

- LLM: **Google Vertex AI → `gemini-3.1-flash-lite`**
- Embeddings: **`text-embedding-004` (Google)**
- Vector DB: **Qdrant (self-hosted, Mumbai)**
- Cache: **Redis**
- Voice: **Sarvam AI (streaming STT + TTS)**
- Backend: **Node.js (Fastify/Express) or Python (FastAPI)**

Region Constraint (NON-NEGOTIABLE)

All components must run in:

```
asia-south1 (Mumbai)
```

SYSTEM ARCHITECTURE

Correct Flow

```
Browser mic
→ Sarvam STT (stream)
→ partial transcript (no waiting)

→ parallel:
   → RAG (Qdrant)
   → LLM trigger

→ Gemini (stream)
→ tokens → TTS (stream)
→ audio plays immediately
```

Forbidden Flow

```
STT → wait → LLM → wait → TTS → play
```

IMPLEMENTATION DETAILS

1. Gemini (Vertex AI) Integration

```
import { GoogleGenAI } from "@google/genai";

const client = new GoogleGenAI({
  vertexai: true,
  project: process.env.GOOGLE_CLOUD_PROJECT,
  location: "asia-south1"
});

const stream = await client.models.generateContentStream({
  model: "gemini-3.1-flash-lite",
  contents: prompt,
  generationConfig: {
    maxOutputTokens: 80,
    temperature: 0.2
  }
});

for await (const chunk of stream) {
  sendToTTS(chunk.text);
}
```

2. RAG System

Constraints

- chunk size: 300–400 tokens
- overlap: 50
- top_k: 2
- context limit: < 1000 tokens

Prompt

```
Answer strictly from context.

Context:
{top_2_chunks}

User:
{query}
```

3. Memory Handling

- last 2–3 turns only

- no full conversation history

4. Caching (Redis)

Cache:

- embeddings
- frequent queries
- final responses

5. Voice Pipeline Rules

- streaming STT
- trigger on partial speech
- streaming LLM
- streaming TTS
- playback starts immediately

6. Backend Rules

- DO NOT use n8n in main flow
- direct API calls only
- reuse persistent connections

LATENCY OPTIMIZATION RULES

- no blocking or sequential execution
- parallel processing
- streaming across STT, LLM, TTS
- minimal prompt size

DELIVERABLES

- working web-based voice assistant
- backend APIs
- RAG pipeline

- latency benchmark report

SUCCESS CRITERIA

- response starts in < 2 seconds
- no silence gaps
- accurate context-based answers
- stable under concurrent usage

FAILURE CONDITIONS

- response delay > 3 seconds
- blocking workflows
- excessive prompt/context
- non-streaming system

HARD CONSTRAINTS

- use only `gemini-3.1-flash-lite`
- no OpenRouter
- no n8n in runtime flow
- no sequential pipeline
- Mumbai region mandatory

FINAL DIRECTIVE

System must behave like real-time conversation:

- user speaks
- system responds within ~1 second

Anything slower fails the task.